Let's Build the GPT Tokenizer (Karpathy): Synthesis

Nov 5, 2026 · @Giáo sư Thịnh

The big picture

A tokenizer is the translation layer between text and the integers a language model actually reads. In this video Karpathy builds the GPT-style tokenizer from scratch and shows that it is a separate stage with its own training set and its own algorithm: byte pair encoding (BPE). This doc is my synthesis from what I know of the video, not a transcript, so treat numbers as approximate.

Why it matters: the model never sees letters or words, only token ids. Many famous LLM weaknesses, such as poor spelling, bad arithmetic, worse performance in some languages, and strange glitches, trace back to how text is cut into tokens.

The path the video takes:

1. Why the character-level approach from the earlier GPT video does not scale.
2. Text as Unicode and bytes (UTF-8).
3. The BPE algorithm, implemented step by step.
4. How GPT-2 and GPT-4 add regex rules to control which merges are allowed.
5. Special tokens, and a look at OpenAI's tiktoken library.
6. SentencePiece, the library behind Llama, and how it differs.
7. Vocabulary size, and a tour of the quirks tokenization causes.

The result: a small library called `minbpe` (a few hundred lines) that trains, encodes, and decodes like the GPT-4 tokenizer.

From text to bytes

The problem with characters: the earlier GPT video used one token per character (65 of them). Real text has many more symbols, and character-level sequences are long, so attention has less text to look at within a fixed context window. We want chunks bigger than one character, and we want to handle every language and emoji.

Unicode code points: Python strings are sequences of Unicode code points, integers that name about 150,000 characters across scripts. `ord('a')` gives 97. You could use code points as tokens, but the vocabulary would be huge and keeps changing as Unicode grows.

UTF-8: an encoding that turns each code point into 1 to 4 bytes. ASCII characters take 1 byte; most other characters take 2 to 4. `'hello'.encode('utf-8')` gives bytes you can turn into a list of integers from 0 to 255.

Encoding	Bytes per character	Why it is or is not used
UTF-8	1 to 4	The standard on the web and the choice for tokenizers: compact for English, backward compatible with ASCII
UTF-16	2 or 4	Wastes space with many zero bytes for ASCII text
UTF-32	4	Fixed width but very wasteful

The idea: start from the raw UTF-8 bytes, so the base vocabulary is exactly 256 symbols and any string can be represented. Then compress the byte sequence by merging frequent pairs into new tokens. That is BPE. The raw byte stream is too long to feed to a transformer directly, which is exactly why we compress it.

Byte pair encoding (BPE)

The idea in one sentence: repeatedly find the most frequent adjacent pair of tokens in the data and replace every occurrence with one new token, until you reach the vocabulary size you want.

A worked example (the small example the video uses): the string `aaabdaaabac`.

Step	Merge	Sequence after	Length
0	none	`aaabdaaabac`	11
1	`aa` becomes Z	`ZabdZabac`	9
2	`ab` becomes Y	`ZYdZYac`	7
3	`ZY` becomes X	`XdXac`	5

The sequence got shorter while the vocabulary grew by one symbol per merge. Note that merge 3 reuses a token made in earlier merges, so tokens can grow into whole words.

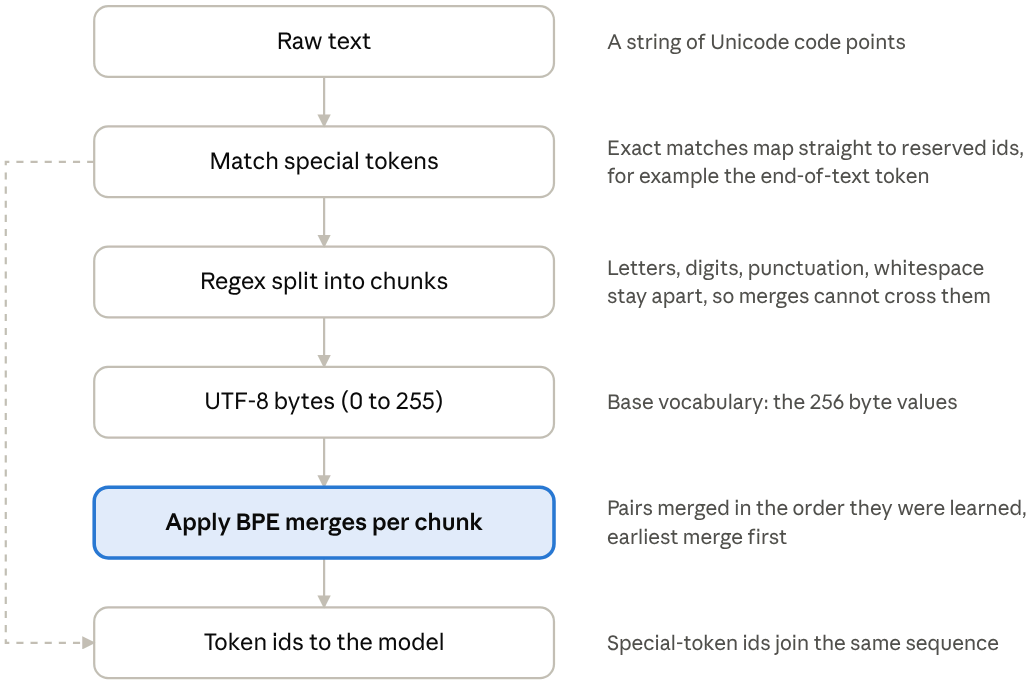
Implementation, as built in the video:

1. **Start:** convert the training text to UTF-8 bytes, as a list of integers 0 to 255.
2. **`get_stats(ids)`:** count how often each adjacent pair occurs.
3. **`merge(ids, pair, idx)`:** scan the list and replace every occurrence of `pair` with the new id `idx`.
4. **Training loop:** to reach a vocabulary of `vocab_size`, do `vocab_size - 256` merges. Each time, find the most common pair, give it the next id (256, 257, ...), merge it everywhere, and record it in a `merges` dictionary mapping the pair to its new id.
5. **Vocab for decoding:** map each id to the bytes it stands for. Bytes 0 to 255 map to themselves, and each merged id maps to the concatenation of its two parts.
6. **`decode(ids)`:** look up each id's bytes, join them, and decode as UTF-8 with `errors='replace'`. The `replace` option is needed because a model can output a sequence of ids that is not valid UTF-8, and decoding should not crash.
7. **`encode(text)`:** convert to bytes, then repeatedly find, among the pairs present, the one with the lowest merge id (the earliest learned merge), and merge it. Stop when no pair in the text exists in `merges`. Applying merges in the order they were learned is what makes encoding match training.

Key property: the tokenizer is trained separately from the language model, on its own dataset. The LLM then trains on the token ids. The training text of the tokenizer decides which languages and formats get efficient (short) tokenizations. Non-English text and code tend to be split into more tokens if they were rare in the tokenizer's training data.

The pipeline at a glance

Text is split, turned into bytes, then merged; special tokens skip BPE



Dashed line: special tokens skip the regex, bytes, and merge steps

tokenization pipeline · 6 steps, 1 bypass

The whole encoding path in one picture: special tokens are pulled out first, the rest is split by a regex, each chunk becomes bytes, and BPE merges shrink each chunk into token ids. Decoding runs the vocabulary lookup backwards.

Regex splitting, GPT-2 and GPT-4, special tokens

The problem with plain BPE: it may merge across unrelated characters. A frequent word followed by punctuation (`dog.` , `dog!` , `dog?`) could produce several different tokens that fuse the word with its punctuation. That wastes vocabulary and mixes semantics.

GPT-2's fix: split first, then merge. A regular expression cuts the text into chunks. Each chunk is tokenized by BPE independently, and the results are concatenated. No merge can ever cross a chunk boundary. The pattern separates:

- contractions (`'s` , `'t` , `'re` , and so on),
- runs of letters (with an optional leading space),
- runs of digits,

- runs of other symbols and punctuation,
- whitespace.

GPT-4 (the `cl100k_base` tokenizer) refines this:

- Contractions match case-insensitively.
- Numbers are limited to chunks of 1 to 3 digits, so very long numbers are not turned into huge tokens.
- Whitespace handling is adjusted, and the vocabulary grows to roughly 100,000 tokens (GPT-2's was 50,257, which is 256 bytes + 50,000 merges + 1 special token).

In the video, Karpathy also notes that OpenAI released the GPT-2 vocabulary and merge files, and that they match the same merge-based idea he implemented, apart from a cosmetic byte remapping. His `minbpe` library includes a `RegexTokenizer` and a `GPT4Tokenizer` that reproduces GPT-4's token ids.

Special tokens

Special tokens are strings that never go through BPE. The tokenizer matches them exactly in the text and maps each to a reserved id.

- `<|endoftext|>` (GPT-2 and later; id 50256 in GPT-2): marks the boundary between documents in training data.
- `<|fim_prefix|>`, `<|fim_middle|>`, `<|fim_suffix|>` (GPT-4 tokenizer): fill-in-the-middle for code.
- `<|endofprompt|>` (GPT-4 tokenizer): marks where a prompt ends.
- Chat markers such as `<|im_start|>` (chat models): delimit turns like user and assistant in a conversation.

Adding a special token is model surgery: the embedding table needs one more row, and the final linear layer needs one more output. Both are initialized for the new id and then trained, commonly while freezing the rest of the model.

tiktoken is OpenAI's library for the inference side only: it loads existing vocabularies (`tiktoken.get_encoding('cl100k_base')`) and encodes fast, but it cannot train a new tokenizer.

SentencePiece and Llama

SentencePiece is Google's library, used by the Llama and Mistral families. Unlike tiktoken it supports both training and inference, and it can use BPE or other algorithms.

The main difference from GPT-style tokenizers:

	GPT-style (tiktoken, `minbpe`)	SentencePiece BPE
Merges happen over	UTF-8 bytes	Unicode code points
Rare characters	Always representable as bytes	Mapped to an unknown token, unless byte fallback is on
Splitting rules	A regex decides the chunks	Built-in options, plus a limit on piece length
Setup	Short, explicit	Many configuration options with legacy defaults

Notable behaviors:

- **Byte fallback:** with `byte_fallback=True` (as Llama uses), code points that were too rare to get their own token are encoded as raw UTF-8 bytes instead of becoming an unknown token. The 256 byte tokens sit in the vocabulary.
- **Whitespace:** spaces are replaced by a visible marker (`▁`), and an optional *dummy prefix* adds a space at the start of the text. This way `world` at the start of a sentence and `world` in the middle get the same token.
- **Special tokens:** typically `<unk>`, `<s>` (beginning of sequence), `</s>` (end of sequence), and optionally a padding token.
- **Normalization:** optional text normalization rules can change the input before tokenizing, which is a risk if you want exact round trips.

Karpathy's verdict (as I recall it): SentencePiece works and is widely used, but its many options and historical defaults are easy to get wrong. When he would train a vocabulary he prefers the cleaner byte-level approach, and the `minbpe` library aims to be that: training that matches the logic of the GPT-4 tokenizer, with the efficiency left to be improved. If you must use SentencePiece, copy the settings of a model you trust (for example Llama's) rather than guessing.

Vocabulary size

Where the vocabulary appears in the model: twice. The input embedding table has one row per token, and the final linear layer has one output per token. Making the vocabulary bigger grows both.

The trade-off:

- **Bigger vocabulary:** each token covers more text, so sequences get shorter and the model sees more text per context window. But each token appears less often in training, so its embedding is trained less, and the output layer costs more compute.
- **Smaller vocabulary:** longer sequences, so less text fits in the context and generation takes more steps.
- **Too large:** rare tokens are barely trained, and unrelated pieces of text get squeezed into one token.

In practice: the vocabulary size is a hyperparameter, and recent models cluster in the tens of thousands to roughly 100,000 or more (GPT-2 about 50,000, Llama 2 32,000, GPT-4 about 100,000).

Extending a trained model's vocabulary: adding tokens after pretraining is possible. Add rows to the embedding table and the output layer, initialize them, and train only the new parts if you want to keep the rest frozen. This is how special tokens and new modalities get added.

Beyond text: the same idea applies to images, audio, and video. Convert the signal into a stream of discrete tokens (or feed it in as soft tokens) and let the same transformer process it. Karpathy points to this as a reason tokenization generalizes well.

Why LLMs behave strangely

The video's last part walks through odd behaviors and ties each to tokenization. The table lists the ones I recall.

Quirk	Tokenization cause
Cannot spell a word, reverse it, or count its letters reliably	A common word is one token, so the model never sees its letters

Quirk	Tokenization cause
Worse at non-English languages	The tokenizer saw less of that language, so sentences split into many more tokens, using up context and giving each token less training signal
Weak at arithmetic	Numbers are cut into arbitrary chunks of digits, so the same digit sequence can be tokenized differently depending on context
GPT-2 was poor at Python	Each indentation space was its own token, wasting context. GPT-4 groups runs of whitespace
Trouble when a prompt ends with a space	A lone trailing space is an unusual token, since normally the space is the start of the next word's token
Strange failure when the text contains the end-of-text token string	The string can be parsed as a special token instead of ordinary text
SolidGoldMagikarp glitch	A string was frequent enough in the tokenizer's data to get its own token, but almost absent from the model's training data, so its embedding was never trained
JSON costs more than YAML	JSON's quotes, braces, and commas produce more tokens for the same data

Common thread: the model's view of text is the token sequence. Anything that depends on characters inside a token, or on consistent digit handling, is harder for it. Anything that makes the tokenizer waste tokens (rare languages, odd whitespace, verbose formats) costs context and money.

Takeaways and a build checklist

Takeaways

1. A tokenizer is its own training stage: it learns merges from a dataset, then the LLM trains on the resulting ids.
2. Byte-level BPE starts from 256 bytes, so any text can be encoded, and merges frequent pairs to shorten sequences.

3. A regex split before merging keeps tokens from crossing letters, digits, punctuation, and whitespace.
4. Special tokens bypass BPE and need matching rows in the embedding and output layers.
5. Many LLM quirks (spelling, arithmetic, non-English performance, whitespace glitches) are tokenization effects.

Practical advice (as I recall Karpathy's recommendations)

- If you can reuse an existing tokenizer such as tiktoken's, do that for inference.
- If you must train one, prefer a byte-level BPE with a regex split, and check it on your own data, including code and your target languages.
- Compare tokens per sentence across languages and formats, because that decides your real context length and cost.
- If you use SentencePiece, copy proven settings rather than guessing, and test the round trip (`decode(encode(text)) == text`).
- Treat user-supplied text carefully if special-token strings can be parsed from it.

Checklist to build your own

- [ ] Encode text to UTF-8 bytes and print them as integers.
- [ ] Write `get_stats` and `merge`, and check them on `aaabdaaabac`.
- [ ] Train for `vocab_size - 256` merges and store the `merges` dictionary.
- [ ] Build the `vocab` (id to bytes) and write `decode` with `errors='replace'`.
- [ ] Write `encode` that applies merges in the order learned.
- [ ] Test the round trip on several texts, including emoji and non-English text.
- [ ] Add the GPT-style regex split, then special tokens.
- [ ] Compare your token ids against tiktoken's `cl100k_base` on sample text.
- [ ] Train on a larger text file and measure the compression ratio (bytes per token).

Where to go next: the `minbpe` repository, the tiktoken library, the *Let's build GPT* video, and then *Let's reproduce GPT-2 (124M)*, which uses the GPT-2 tokenizer.